

ゼロ知識証明方式別解説

構成と実現した性質

Groth16, PLONK, BBS+ 署名, Bulletproofs, One-out-of-Many proof,
ZK-STARK, Ligerio, Aurora, Virgo, zkVM/Jolt の整理

本資料は、各 ZKP 方式を「どのような構成で、何を実現したか」という観点から整理した補足解説である。性能比較表では見えにくい、算術化、コミットメント、対話プロトコル、非対話化、trusted setup、一般プログラムへの適用方法を中心に説明する。

Contents

1 本資料の読み方：ZKP 方式を四層で捉える	3
2 方式別クイックマップ	3
3 汎用計算証明における基本変換	4
4 Groth16：QAP とペアリングで極小証明を実現する	5
4.1 何を証明する方式か	5
4.2 構成の流れ	5
4.3 この構成で実現したこと	6
4.4 一般プログラムへの適用	6
5 PLONK 系：Plonkish 制約と permutation argument で柔軟性を高める	6
5.1 何を証明する方式か	6
5.2 構成の流れ	6
5.3 この構成で実現したこと	7
5.4 一般プログラムへの適用	7
6 BBS+ 署名：署名知識証明で選択的開示を実現する	7
6.1 何を証明する方式か	7
6.2 構成の流れ	8
6.3 この構成で実現したこと	8
6.4 一般プログラムへの適用	8
7 Bulletproofs：内積証明で trusted setup 不要の範囲証明を実現する	8
7.1 何を証明する方式か	8
7.2 構成の流れ	8
7.3 この構成で実現したこと	9
7.4 一般プログラムへの適用	9
8 One-out-of-Many proof：集合内の 1 要素を秘匿して証明する	9
8.1 何を証明する方式か	9
8.2 構成の流れ	9
8.3 この構成で実現したこと	10
8.4 一般プログラムへの適用	10
9 ZK-STARK：実行トレース、AIR、FRI で透明かつスケーラブルな証明を実現する	10
9.1 何を証明する方式か	10
9.2 構成の流れ	10

9.3	この構成で実現したこと	11
9.4	一般プログラムへの適用	11
10	Ligero : PCP/MPC-in-the-head で公開鍵暗号なしの軽量 ZK を実現する	11
10.1	何を証明する方式か	11
10.2	構成の流れ	11
10.3	この構成で実現したこと	12
10.4	一般プログラムへの適用	12
11	Aurora : R1CS を透明セットアップ型 IOP へ接続する	12
11.1	何を証明する方式か	12
11.2	構成の流れ	12
11.3	この構成で実現したこと	12
11.4	一般プログラムへの適用	12
12	Virgo : GKR と多項式委譲で構造化回路の簡潔検証を実現する	13
12.1	何を証明する方式か	13
12.2	構成の流れ	13
12.3	この構成で実現したこと	13
13	zkVM と Jolt : 一般プログラムを命令実行として証明する	13
13.1	何を証明する方式か	13
13.2	構成の流れ	14
13.3	この構成で実現したこと	14
13.4	注意点	14
14	方式別に見た「何を実現したか」	14
15	一般プログラムへの適用観点での比較	15
16	方式選定の実務的な読み替え	16
17	まとめ	17
	参考文献	18
	索引	19

1 本資料の読み方：ZKP 方式を四層で捉える

ZKP の各方式は、単なる「証明サイズ」や「検証時間」の違いだけでは理解しにくい。むしろ、方式ごとに、次の四つの層をどのように構成しているかを見ると、何を実現したのかが明確になる。

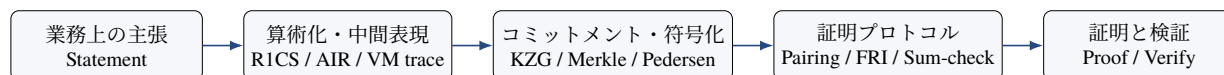


Figure 1: ZKP 方式を理解するための四層モデル

第一の層は、**算術化**または証明対象の表現である。自然言語の主張や通常のプログラムは、そのままでは ZKP で扱えない。そこで、**R1CS**、**QAP**、**Plonkish 制約**、**AIR**、**layered arithmetic circuit**、**VM 実行トレース**などへ変換する。

第二の層は、コミットメントまたは符号化である。証明者は、witness や多項式、実行トレースをそのまま公開せず、KZG コミットメント、Merkle commitment、Pedersen commitment、Reed-Solomon 符号化などを通じて、検証可能な形に固定する。

第三の層は、証明プロトコルである。Groth16 ではペアリング方程式、Bulletproofs では内積証明、STARK では FRI、Virgo では sum-check/GKR、One-out-of-Many では Sigma プロトコルが中心になる。

第四の層は、非対話化と検証である。多くの方式では Fiat-Shamir 変換を使って、検証者のランダムチャレンジをハッシュから生成し、非対話型証明に変換する。

要点

ZKP 方式の違いは、「同じ statement をどれだけ速く証明できるか」だけではない。そもそも、statement をどの中間表現へ落とすか、どのコミットメントで固定するか、どの確率検査で正しさを保証するかが異なる。この違いが、trusted setup、証明サイズ、検証時間、ポスト量子性、開発容易性の差として現れる。

2 方式別クイックマップ

Table 1: 各 ZKP 方式の構成と実現した性質

方式	主な証明対象	構成の核心	実現した性質
Groth16	汎用計算、R1CS/QAP	R1CS を QAP へ変換し、trusted setup で生成した SRS とペアリングにより多項式関係を検証する。	3つの群要素からなる極小証明と高速検証。安定した固定回路に強い。 ^[1]
PLONK 系	汎用計算、Plonkish 制約	行列状の witness 表、gate 制約、permutation argument、多項式コミットメントを組み合わせる。	ユニバーサル/更新可能 SRS、custom gate、lookup による柔軟な回路表現。 ^[2]

方式	主な証明対象	構成の核心	実現した性質
BBS+ 署名	属性証明、選択的開示	複数メッセージ署名と署名知識証明を組み合わせ、署名済み属性の一部だけを開示する。	選択的開示、unlinkability、匿名資格証明。汎用計算証明ではない。 ^{[3][4]}
Bulletproofs	範囲証明、内積関係	Pedersen commitment、ビット分解、内積証明を組み合わせる。	trusted setup 不要、レンジプルーフの短縮、証明サイズが witness サイズに対して対数的。 ^[5]
One-out-of-Many	匿名集合所属	コミットメント列のうち少なくとも 1 つが 0 になることを Sigma プロトコルで示す。	集合内のどの要素かを秘匿したまま所属を証明し、通信量を対数的に抑える。 ^[6]
ZK-STARK	大規模計算、実行トレース	計算を AIR と実行トレースにし、低次数検査と FRI、Merkle commitment で検証する。	透明セットアップ、スケーラビリティ、ポスト量子性を期待し得る構成。 ^[7]
Ligero	NP、検証回路	MPC-in-the-head/interactive PCP、Reed-Solomon 符号、ハッシュコミットメントを組み合わせる。	trusted setup や公開鍵暗号なしで、平方根オーダーの通信量を実現。 ^[8]
Aurora	R1CS	R1CS を IOP/AHP 的に検査し、透明セットアップと軽量暗号で構成する。	R1CS と透明セットアップを接続し、plausibly post-quantum な zkSNARK 系構成を実現。 ^[9]
Virgo	layered arithmetic circuit	GKR/sum-check と透明な多項式委譲を組み合わせる。	構造化回路に対して、trusted setup なしで簡潔な検証を実現。 ^[10]
zkVM/Jolt	一般プログラム実行	プログラムを RISC-V 等の命令列に変換し、実行トレースまたは lookup で命令実行を証明する。	通常のプロプログラムに近い開発体験。Jolt は ISA 依存の巨大 lookup table を使う。 ^{[11][12]}

3 汎用計算証明における基本変換

汎用計算証明では、一般的なプログラムをそのまま証明するのではなく、計算モデルに合わせて変換する。代表的な変換は次の通りである。

Table 2: 一般プログラムを ZKP へ載せる代表的な変換

変換先	考え方	主な方式
算術回路/R1CS	加算・乗算制約の集合に変換する。分岐は selector、比較は range check、ループは固定長展開または上限制約で扱う。	Groth16、Aurora
Plonkish 制約	行と列の表として計算を表し、gate 制約、copy 制約、lookup で処理を分解する。	PLONK 系、Halo2 系
AIR/実行トレース	各時刻の状態を表に並べ、隣接行の遷移制約と境界制約で正しい実行を示す。	ZK-STARK、STARK 系 zkVM
layered arithmetic circuit	回路を層構造にし、各層間の評価関係を sum-check/GKR 型に検査する。	Virgo
VM 実行トレース	Rust/C/C++ などを RISC-V 等へコンパイルし、命令実行が仕様通りであることを証明する。	zkVM、Jolt、RISC Zero、SP1 等
用途特化関係	範囲、署名知識、集合所属など、限定された statement に最適化する。	BBS+ 署名、Bulletproofs、One-out-of-Many

有限体上の制約では、通常のプログラムの意味が自動的に保存されるわけではない。たとえば、有限体には通常の整数としての大小関係がないため、比較にはビット分解や range check が必要になる。分岐は片方だけを「実行」するのではなく、両方の候補値を作り、Boolean selector で選ぶ形にする。

$$b(b-1) = 0, \quad y = b \cdot y_1 + (1-b) \cdot y_0$$

これは、ZKP で一般プログラムを扱う際の根本的な制約である。各方式の構成は、この制約をどのように効率化するかの違いとして理解できる。

4 Groth16 : QAP とペアリングで極小証明を実現する

4.1 何を証明する方式か

Groth16 は、R1CS/QAP で表現された汎用計算の充足性を、非常に短い非対話型証明で示す方式である。Groth16 の原論文は、ペアリングベースの preprocessing SNARK として、証明が 3 つの群要素で済み、検証は単一の pairing product equation で行えることを示した。^[1]

4.2 構成の流れ



Figure 2: Groth16 の構成イメージ

まず、公開入力を x 、秘密入力を w 、中間変数を含むベクトルを $z = (1, x, w, \dots)$ とする。R1CS は次の形の制約集合である。

$$(Az) \circ (Bz) = Cz$$

ここで $\circ$ は成分ごとの積である。R1CS はさらに QAP へ変換される。QAP では、制約充足性を次のような多項式の割り切り条件に変換する。

$$A_z(X)B_z(X) - C_z(X) = H(X)t(X)$$

Groth16 の setup では、秘密値 τ や補助的な秘密値を使って、これらの多項式を τ で評価した形の SRS を生成する。証明者は、 $A_z(\tau)$ 、 $B_z(\tau)$ 、 $H(\tau)$ などに対応する群要素を組み合わせて証明を作る。検証者は、ペアリングの双線形性を使って、多項式関係が成立していることを少数の群演算で確認する。

4.3 この構成で実現したこと

Groth16 が実現した主要な成果は、**証明サイズの極小化**と**高速検証**である。証明が3つの群要素で済むため、オンチェーン検証や大量検証に強い。検証者は回路全体を再実行せず、ペアリング方程式を確認するだけでよい。

この簡潔性は、SRS に隠された τ を通じて「多項式全体を開かずに、特定点での関係を検証できる」ことから生じる。つまり、証明者は多項式関係を満たしていることを、値そのものや witness を公開せずに、群要素として提示する。

4.4 一般プログラムへの適用

Groth16 で一般プログラムを扱うには、プログラムを固定サイズの R1CS へ変換する必要がある。分岐は selector で表し、ループは最大回数まで展開するか、inactive flag で無効化する。動的メモリや可変長処理は扱いにくい。

したがって、Groth16 は、回路が安定しており、証明サイズと検証時間を最優先する用途に適する。一方、業務ロジックが頻繁に変わる場合、回路変更に伴う再 setup や鍵管理が負担になる。

5 PLONK 系：Plonkish 制約と permutation argument で柔軟性を高める

5.1 何を証明する方式か

PLONK は、汎用算術回路に対する SNARK であり、Groth16 のような回路固有 setup の負担を軽減するため、ユニバーサルかつ更新可能な SRS を利用できる方向を示した方式である。PLONK の原論文は、universal SNARK と fully succinct verification を提示した。^[2]

5.2 構成の流れ

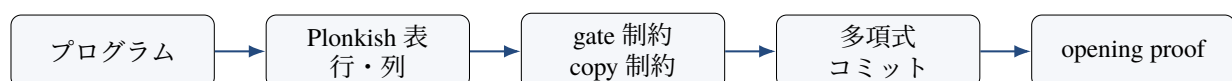


Figure 3: PLONK 系の構成イメージ

Plonkish 制約では、witness を「行」と「列」の表として配置する。典型的な 1 行の制約は次のように書ける。

$$q_M ab + q_L a + q_R b + q_O c + q_C = 0$$

ここで a, b, c はワイヤ値、 q_M, q_L, q_R, q_O, q_C は selector である。さらに、同じ値が別のセルにコピーされていることを **permutation argument** で示す。

PLONK 系の重要な発展は、**custom gate** と **lookup argument** である。たとえば、8 ビットの XOR を有限体演算だけで表すと多数の制約が必要になるが、lookup table を使えば次のように扱える。

$$(a, b, c) \in T_{\text{xor}}, \quad c = a \oplus b$$

Plookup は、コミットされた多項式の値が、あらかじめ定めたテーブルに含まれることを証明する手法として提案された。^[13]

5.3 この構成で実現したこと

PLONK 系が実現したのは、**SNARK の簡潔性を保ちながら、回路表現と setup 運用を柔軟にすること** である。Groth16 では回路ごとの setup が問題になりやすいが、PLONK 系では汎用的な SRS を利用できる構成が多い。また、custom gate や lookup により、ハッシュ、範囲チェック、楕円曲線演算、VM 命令などを効率よく表現できる。

5.4 一般プログラムへの適用

PLONK 系では、一般プログラムを直接証明するのではなく、Plonkish 表または zkVM の命令トレースへ変換する。lookup を多用することで、CPU 命令、ビット演算、range check、テーブル参照を効率化できる。Jolt のような VM 向け手法は、この lookup 中心の思想をさらに推し進めている。

ただし、PLONK 系は方式のバリエーションが多い。KZG を使う場合はペアリング仮定と SRS が関係し、FRI 系のコミットメントを使う場合は証明サイズや検証コストが異なる。したがって、「PLONK 系」と一括りにするより、具体的な多項式コミットメントと SRS 管理を確認する必要がある。

6 BBS+ 署名：署名知識証明で選択的開示を実現する

6.1 何を証明する方式か

BBS+ 署名、標準化文脈では BBS 署名は、複数メッセージ署名と署名の知識証明を組み合わせ、署名済み属性の一部だけを開示できるようにする方式である。IETF CFRG の BBS 署名ドラフトは、BBS を複数メッセージ署名方式として説明し、署名の知識証明を行いながら任意の部分集合を選択的に開示できるとする。^[3] W3C Data Integrity BBS Cryptosuites も、BBS 署名が選択的開示と unlinkable proofs を直接提供すると説明している。^[4]

6.2 構成の流れ

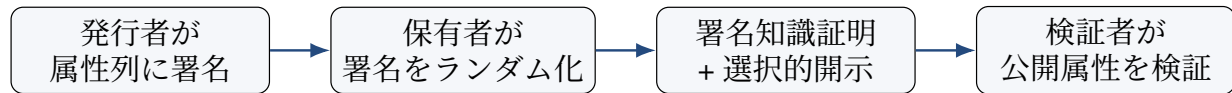


Figure 4: BBS+ 署名による選択的開示の構成イメージ

開示するメッセージ集合を m_D 、秘匿するメッセージ集合を m_H 、署名を σ 、発行者公開鍵を pk_I とすると、証明者は概略的に次を証明する。

$$\exists \sigma, m_H \quad \text{s.t.} \quad \text{Verify}(pk_I, m_D \parallel m_H, \sigma) = 1$$

検証者は、開示された属性 m_D が正当な署名済み資格情報の一部であることを確認できるが、 m_H や元署名 σ を知る必要はない。

6.3 この構成で実現したこと

BBS+ 署名が実現したのは、**属性単位のプライバシー制御**である。通常の署名では、署名対象メッセージ全体を見せなければ署名検証できない。BBS+ 署名では、複数属性をまとめて署名し、提示時には必要な属性だけを開示できる。

さらに、証明提示ごとに署名をランダム化できるため、同じ credential に由来する複数提示をリンクしにくくできる。この性質は、年齢確認、資格証明、会員資格、従業員属性、KYC 済み証明などに適する。

6.4 一般プログラムへの適用

BBS+ 署名は汎用計算証明ではない。たとえば「生年月日から今日時点で 20 歳以上であること」を完全に秘匿して証明するには、単なる選択的開示だけでは足りない場合がある。この場合は、発行者が「20 歳以上」という派生属性に署名するか、署名検証と日付比較を SNARK/STARK の回路内で証明する必要がある。

7 Bulletproofs：内積証明で trusted setup 不要の範囲証明を実現する

7.1 何を証明する方式か

Bulletproofs は、短い非対話型ゼロ知識証明であり、特に Pedersen commitment された値が範囲内にあることを示す **range proof** に強い。原論文は、trusted setup を必要とせず、証明サイズが witness サイズに対して対数的であることを示している。^[5]

7.2 構成の流れ

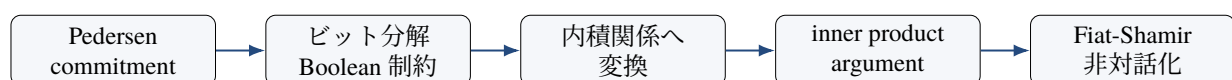


Figure 5: Bulletproofs の構成イメージ

典型的には、値 v を Pedersen commitment で隠す。

$$C = g^v h^r$$

そして、次の範囲条件を証明する。

$$0 \leq v < 2^n$$

範囲証明では、 v をビット列に分解し、各ビットが 0 または 1 であること、ビット列が v と整合することを示す。Bulletproofs は、この関係を内積関係へ変換し、内積証明によって対数サイズに圧縮する。

$$\langle \mathbf{a}, \mathbf{b} \rangle = c$$

7.3 この構成で実現したこと

Bulletproofs が実現したのは、**trusted setup なしで実用的な短い範囲証明を構成すること**である。SNARK ほど検証は軽くないが、setup ceremony を避けながら、金額、残高、担保率、スコアなどの数値条件を秘匿して証明できる。

また、複数の range proof を集約できるため、秘匿取引の複数出力や、複数の数値属性の同時証明に適する。

7.4 一般プログラムへの適用

Bulletproofs は一般算術回路にも拡張可能だが、主戦場は範囲証明と内積関係である。任意の大規模プログラム実行を証明する場合、Groth16、PLONK、STARK、zkVM の方が自然である。Bulletproofs は、「特定の数値が条件を満たす」ことを示す用途で強い。

8 One-out-of-Many proof : 集合内の 1 要素を秘匿して証明する

8.1 何を証明する方式か

One-out-of-Many proof は、公開されたコミットメント列のうち、少なくとも 1 つについて特定の開封情報を知っていることを、その位置を明かさずに証明する方式である。Groth と Kohlweiss は、少なくとも 1 つが 0 に開くコミットメント列について、どれが該当するかを秘匿する 3 手の public coin special honest verifier zero-knowledge な Sigma プロトコルを構成した。通信量はコミットメント数に対して対数的である。^[6]

8.2 構成の流れ



Figure 6: One-out-of-Many proof の構成イメージ

典型的な statement は次の形である。

$$\exists j, r_j \quad \text{s.t.} \quad C_j = \text{Com}(0; r_j)$$

Pedersen commitment なら、

$$C_j = g^0 h^{r_j} = h^{r_j}$$

である。検証者は、証明者がどこか 1 つの開封情報を知っていることを確認するが、どの C_j なのかは分らない。

8.3 この構成で実現したこと

One-out-of-Many proof が実現したのは、**匿名集合所属**である。リング署名的な用途、匿名支払い、匿名アクセス、プライバシー保護型のメンバーシップ証明に使える。

この方式の本質は、「集合内の 1 要素を知っている」ことを、線形サイズではなく対数的な通信量で示せる点にある。すべてのコミットメントについて個別に証明する必要はない。

8.4 一般プログラムへの適用

One-out-of-Many proof は汎用計算証明ではない。集合所属や匿名性に特化している。したがって、複雑な業務ロジックを証明するには、SNARK/STARK 等と組み合わせる必要がある。

9 ZK-STARK : 実行トレース、AIR、FRI で透明かつスケーラブルな証明を実現する

9.1 何を証明する方式か

ZK-STARK は、Scalable Transparent Argument of Knowledge の略であり、透明セットアップ、スケーラビリティ、ポスト量子性を重視する証明方式である。ZK-STARK の論文は、transparent ZK system であり、verification が大規模計算に対して大きく高速化する構成を提示した。^[7]

9.2 構成の流れ



Figure 7: ZK-STARK の構成イメージ

STARK では、プログラム実行をトレース表として表す。各行が時刻 i の状態、各列がレジスタ、メモリ、フラグなどを表す。

$$T = \begin{bmatrix} T_{0,1} & T_{0,2} & \cdots & T_{0,w} \\ T_{1,1} & T_{1,2} & \cdots & T_{1,w} \\ \vdots & \vdots & & \vdots \\ T_{n,1} & T_{n,2} & \cdots & T_{n,w} \end{bmatrix}$$

AIR では、各ステップの正しさを遷移制約で表す。

$$F_j(T_i, T_{i+1}) = 0$$

さらに、初期状態、終了状態、公開出力などを境界制約で固定する。

$$B_{\text{init}}(T_0, x) = 0, \quad B_{\text{out}}(T_n, y) = 0$$

証明者は、このトレースが低次数多項式として整合していることを示す。FRI は、ある関数が低次数多項式に近いことを効率的に検査するためのプロトコルである。Merkle commitment を用いることで、検証者はトレース全体を読むことなく、ランダムに選んだ位置だけを確認できる。

9.3 この構成で実現したこと

ZK-STARK が実現したのは、**trusted setup を避けながら、大規模計算の正しさを検証できること**である。ペアリングや楕円曲線に依存せず、主にハッシュ関数と多項式検査に基づくため、ポスト量子性を期待し得る方式として位置づけられる。

STARK は、長い実行トレースを扱えるため、VM 実行、ブロックチェーンの状態遷移、バッチ処理、再帰的証明と相性がよい。一方、証明サイズは Groth16 のようなペアリングベース SNARK より大きくなりやすい。

9.4 一般プログラムへの適用

一般プログラムは、STARK VM や Cairo のような専用 VM、または RISC-V 系 zkVM を通じて実行トレースに変換される。分岐やループは、プログラムカウンタと状態遷移として表現できるため、静的な回路展開より自然に扱える場合がある。ただし、メモリ整合性、命令デコード、範囲制約、ハッシュ関数の AIR 化は高度な設計を要する。

10 Ligero : PCP/MPC-in-the-head で公開鍵暗号なしの軽量 ZK を実現する

10.1 何を証明する方式か

Ligero は、NP に対する軽量な ZK argument であり、通信量が検証回路サイズの平方根に比例する。拡張版論文は、任意の衝突困難ハッシュ関数に基づき、ランダムオラクルモデルでは trusted setup や公開鍵暗号を必要としない非対話型 zk-SNARK として利用できると説明している。^[8]

10.2 構成の流れ

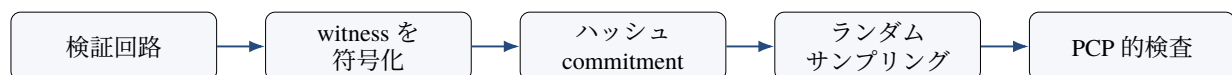


Figure 8: Ligero の構成イメージ

Ligero では、witness や計算結果を符号化し、検証者がランダムに選んだ一部を検査する。直感的には、証明者が大きな表を作り、その表が正しい計算を符号化していることを、検証者が少数のクエリで確認する。

この構成は、MPC-in-the-head 的な考え方と interactive PCP を組み合わせる。証明者は、計算が正しいことを示すための符号化された証拠をコミットし、検証者はランダムな位置を開かせる。

10.3 この構成で実現したこと

Ligero が実現したのは、**trusted setup** や公開鍵暗号を避けた、比較的単純で軽量な **ZK argument** である。ペアリングや楕円曲線に依存しないため、透明性やポスト量子性を重視する文脈で意味を持つ。

ただし、証明サイズは Groth16 のような極小証明ではない。通信量は平方根オーダーであるため、大規模計算では STARK や他の succinct proof との比較が必要になる。

10.4 一般プログラムへの適用

Ligero は、一般プログラムを直接実行する VM ではない。一般プログラムは、まず検証回路または算術回路に変換され、その回路の充足性を Ligero で証明する。したがって、開発容易性は回路化 frontend に依存する。

11 Aurora : R1CS を透明セットアップ型 IOP へ接続する

11.1 何を証明する方式か

Aurora は、R1CS に対する透明な succinct argument である。Aurora の論文は、R1CS 向け zkSNARK を設計・実装・評価し、transparent setup、plausibly post-quantum secure、lightweight cryptography を特徴とすると説明している。証明サイズは $O(\log^2 n)$ 、証明生成は $O(n \log n)$ 体演算、検証は $O(n)$ とされる。^[9]

11.2 構成の流れ

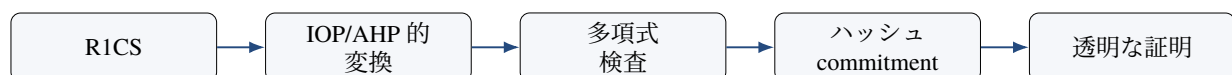


Figure 9: Aurora の構成イメージ

Aurora の意義は、Groth16 などでも広く使われる R1CS という表現を、ペアリングベースではなく、透明な IOP/ハッシュ系の証明へ接続した点にある。R1CS の充足性を、多項式や符号語の検査問題へ変換し、検証者が全体を読まずに確率的に確認できるようにする。

11.3 この構成で実現したこと

Aurora が実現したのは、**R1CS 互換性と透明セットアップの両立**である。既存の R1CS ベース開発資産や理論を活かしつつ、trusted setup やペアリングへの依存を避ける方向を示した。

Groth16 と比べると、証明サイズや検証コストでは不利になる場合があるが、setup 透明性とポスト量子性を期待し得る点異なる。

11.4 一般プログラムへの適用

一般プログラムは、Groth16 と同様に R1CS へ落とす必要がある。そのため、分岐、ループ、比較、メモリなどの取り扱いは、回路化 frontend の品質に依存する。Aurora 自体は、R1CS へ落とされた後の証明方式である。

12 Virgo : GKR と多項式委譲で構造化回路の簡潔検証を実現する

12.1 何を証明する方式か

Virgo は、layered arithmetic circuits 向けの transparent zero knowledge argument である。論文は、trusted setup なしで、 D 層、入力数 n 、ゲート数 C の回路に対し、証明者時間 $O(C + n \log n)$ 、証明サイズ $O(D \log C + \log^2 n)$ を実現し、構造化回路では検証時間も $O(D \log C + \log^2 n)$ になると説明している。^[10]

12.2 構成の流れ

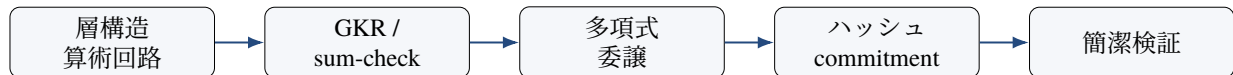


Figure 10: Virgo の構成イメージ

GKR 型プロトコルでは、回路の出力評価が正しいことを、各層の評価関係へ順に還元していく。sum-check プロトコルにより、多変数多項式の総和に関する主張を、少数のランダム点での評価へ圧縮する。

概念的には、次のような主張を順に小さくしていく。

$$\sum_{u \in \{0,1\}^k} f(u) = v$$

この総和主張を、ランダムチャレンジを使って、より小さい評価主張に還元する。Virgo は、この流れを透明な多項式委譲と組み合わせる。

12.3 この構成で実現したこと

Virgo が実現したのは、**層構造を持つ計算に対する trusted setup 不要の簡潔検証**である。Merkle tree、ハッシュ木、繰り返し構造、層ごとのデータ処理など、構造が明確な計算では強みを持つ。一方、複雑な制御フローや動的メモリを含む一般プログラムを直接 Virgo に載せるのは容易ではない。VM 化や回路変換を経由する必要がある。

13 zkVM と Jolt : 一般プログラムを命令実行として証明する

13.1 何を証明する方式か

zkVM は、通常のプログラムを仮想マシン上で実行し、その実行が正しいことを ZKP で証明するための設計である。RISC Zero は、zkVM により任意の Rust code の正しい実行を証明できると説明している。^[12]

Jolt は、VM 向け SNARK の frontend 技術であり、命令実行を巨大な lookup table への参照として扱う。Jolt の論文は、ISA に依存する巨大な lookup table への lookup を中心に回路を構成し、その正当性を Lasso という lookup argument で証明すると説明している。^[11]

13.2 構成の流れ



Figure 11: zkVM/Jolt の構成イメージ

zkVM の statement は、概略的に次のように書ける。

$$\exists T, w \quad \text{s.t.} \quad \text{Exec}_{\text{VM}}(P, x, w) = y$$

ここで P はプログラム、 x は公開入力、 w は秘密入力、 y は公開出力、 T は実行トレースである。より厳密には、プログラムハッシュ、命令デコード、メモリ整合性、公開入出力、終了状態が制約化される。

Jolt 型では、各命令の正しさを次のような lookup で扱う。

$$(\text{opcode}, a, b, c) \in T_{\text{ISA}}$$

たとえば、ADD 命令なら $c = a + b \pmod{2^{32}}$ が、ISA 依存の lookup table に含まれていることを示す。

13.3 この構成で実現したこと

zkVM が実現したのは、**ZKP 開発を通常のプログラミングに近づけること**である。専用回路を手で書くのではなく、Rust や C/C++ をコンパイルして証明できる。これにより、AI 推論、スマートコントラクト検証、外部計算証明、データ処理証明などの開発負担が下がる。

Jolt 型が実現したのは、**VM 命令実行を lookup 中心に整理すること**である。CPU 命令の多くは有限体上の低次数式だけでは効率的に表しにくい。lookup を使えば、命令の意味を巨大だが構造化された表として扱える。

13.4 注意点

zkVM は一般プログラムを扱いやすくするが、証明対象になるのは VM 内の決定的な実行である。外部 API、現在時刻、ファイル、ネットワーク、センサ値は、ホスト側から入力として与えられる。これらの真正性を証明するには、署名、ハッシュ、Merkle root、タイムスタンプなどをプログラム内で検証する必要がある。

14 方式別に見た「何を実現したか」

Table 3: 構成と実現した性質の対応

構成上の工夫	実現した性質	代表方式
QAP + pairing + SRS	極小証明、高速検証、固定回路の効率的証明	Groth16

構成上の工夫	実現した性質	代表方式
Plonkish 制約 + permutation argument	回路表現の柔軟化、copy 制約の簡潔化、ユニバーサル SRS	PLONK 系
複数メッセージ署名 + 署名知識証明	属性の選択的開示、unlinkability、匿名資格証明	BBS+ 署名
Pedersen commitment + inner product argument	trusted setup 不要の短い範囲証明、集約可能性	Bulletproofs
Sigma protocol + コミットメント列	集合内の 1 要素を秘匿した匿名所属証明	One-out-of-Many proof
AIR + FRI + Merkle commitment	透明セットアップ、大規模計算、ポスト量子性を期待し得る証明	ZK-STARK
PCP/MPC-in-the-head + hash commitment	公開鍵暗号なし、平方根通信量、trusted setup 不要	Ligero
R1CS + IOP + 透明 setup	R1CS 互換性と透明性の両立	Aurora
GKR/sum-check + 多項式委譲	構造化回路の簡潔検証、trusted setup 不要	Virgo
ISA trace + lookup argument	一般プログラム実行の証明、開発容易性向上	zkVM/Jolt

15 一般プログラムへの適用観点での比較

Table 4: 一般プログラムを扱う際の方式別取り扱い

方式	一般プログラムの扱い	開発容易性	主な注意点
Groth16	プログラムを R1CS/QAP へ変換。ループは固定長展開、分岐は selector で表す。	低-中	回路変更時の setup、制約漏れ
PLONK 系	Plonkish 表、custom gate、lookup、VM frontend で扱う。	中	実装差、多項式コミットメント選択
BBS+ 署名	一般プログラムではなく、署名済み属性の選択的開示に特化。	属性証明なら高	失効管理、属性 schema、派生属性
Bulletproofs	範囲証明・内積関係に強い。一般回路も可能だが主用途ではない。	用途特化なら高	検証時間、非 PQ
One-out-of-Many	集合所属・匿名性に特化。	用途特化なら中	汎用計算ではない

方式	一般プログラムの扱い	開発容易性	主な注意点
ZK-STARK	プログラムを実行トレース/AIR、または STARK VM に変換。	zkVM なら中-高	証明サイズ、AIR 設計
Ligero	検証回路へ変換し、符号化とサンプリングで証明。	中-低	平方根通信量、frontend 依存
Aurora	R1CS へ変換して透明 IOP で証明。	中	R1CS 化の正しさ、検証コスト
Virgo	layered arithmetic circuit へ変換。構造化計算なら中構造化計算に向く。	構造化計算なら中	一般コードの層化が難しい
zkVM/Jolt	RISC-V 等の ISA 実行として扱い、命令実行を制約または lookup で証明。	高	汎用性の代償として証明コストが重い

一般プログラムに ZKP を適用する場合、最も大きな実装リスクは暗号方式そのものではなく、**変換の正しさ**である。たとえば、年齢確認の statement であっても、発行者署名、属性名、schema、現在日付、失効状態、検証者 nonce、タイムゾーン、境界条件をすべて正しく公開入力または witness として扱う必要がある。

16 方式選定の実務的な読み替え

方式を選ぶときは、まず「どの構成が最も高性能か」ではなく、「どの構成が証明したい statement に自然か」を考えるべきである。

Table 5: 目的から見た方式選定

目的	自然な構成	候補方式
証明サイズを最小化したい	QAP + pairing + trusted setup	Groth16
回路を頻繁に変えたい	ユニバーサル SRS + Plonkish 制約	PLONK 系
属性の一部だけ開示したい	複数メッセージ署名 + 署名知識証明	BBS+ 署名
金額やスコアの範囲だけ示したい	Pedersen commitment + range proof	Bulletproofs
集合内メンバーであることだけ示したい	コミットメント列 + 匿名集合所属証明	One-out-of-Many proof
trusted setup を避け、大規模計算を証明したい	AIR + FRI + Merkle commitment	ZK-STARK
公開鍵暗号を避けた軽量 ZK が欲しい	PCP/MPC-in-the-head + hash commitment	Ligero
R1CS 資産を透明 setup へ接続したい	R1CS + IOP/AHP	Aurora
構造化回路を簡潔に検証したい	GKR/sum-check + 多項式委譲	Virgo

目的	自然な構成	候補方式
通常のコードに近い形で証明したい	ISA 実行トレース + lookup/AIR	zkVM/Jolt

17 まとめ

各 ZKP 方式は、異なる構成上の工夫によって異なる性質を実現している。

- **Groth16** は、R1CS を QAP に変換し、trusted setup とペアリングを用いることで、極小証明と高速検証を実現した。
- **PLONK 系** は、Plonkish 制約、permutation argument、custom gate、lookup により、SNARK の柔軟性と開発容易性を高めた。
- **BBS+ 署名** は、署名知識証明により、署名済み属性の選択的開示と unlinkability を実現した。
- **Bulletproofs** は、Pedersen commitment と内積証明により、trusted setup 不要の短い範囲証明を実現した。
- **One-out-of-Many proof** は、Sigma プロトコルにより、匿名集合所属を対数通信で実現した。
- **ZK-STARK** は、AIR、FRI、Merkle commitment により、透明セットアップ、大規模計算、ポスト量子性を期待し得る証明を実現した。
- **Ligero** は、PCP/MPC-in-the-head とハッシュにより、公開鍵暗号や trusted setup を避けた軽量 ZK を実現した。
- **Aurora** は、R1CS を透明な IOP 系証明へ接続し、R1CS 互換性と透明性の両立を実現した。
- **Virgo** は、GKR/sum-check と多項式委譲により、構造化回路の簡潔検証を実現した。
- **zkVM/Jolt** は、一般プログラムを ISA 実行として証明し、ZKP 開発を通常のソフトウェア開発に近づけた。

したがって、ZKP 方式の比較は、万能な最良方式を探す作業ではない。重要なのは、証明したい statement、秘匿したい witness、許容できる setup、必要な検証コスト、長期安全性、実装成熟度を踏まえて、適切な構成を選ぶことである。

参考文献

- [1] Jens Groth, “On the Size of Pairing-based Non-interactive Arguments,” IACR Cryptology ePrint Archive, Report 2016/260. <https://eprint.iacr.org/2016/260>
- [2] Ariel Gabizon, Zachary J. Williamson, Oana Ciobotaru, “PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge,” IACR Cryptology ePrint Archive, Report 2019/953. <https://eprint.iacr.org/2019/953>
- [3] IETF CFRG, “The BBS Signature Scheme,” draft-irtf-cfrg-bbs-signatures-10, 2026. <https://datatracker.ietf.org/doc/draft-irtf-cfrg-bbs-signatures/>
- [4] W3C, “Data Integrity BBS Cryptosuites v1.0.” <https://www.w3.org/TR/vc-di-bbs/>
- [5] Benedikt Bünz et al., “Bulletproofs: Short Proofs for Confidential Transactions and More,” IACR Cryptology ePrint Archive, Report 2017/1066. <https://eprint.iacr.org/2017/1066>
- [6] Jens Groth and Markulf Kohlweiss, “One-out-of-Many Proofs: Or How to Leak a Secret and Spend a Coin,” IACR Cryptology ePrint Archive, Report 2014/764. <https://eprint.iacr.org/2014/764>
- [7] Eli Ben-Sasson et al., “Scalable, transparent, and post-quantum secure computational integrity,” IACR Cryptology ePrint Archive, Report 2018/046. <https://eprint.iacr.org/2018/046>
- [8] Scott Ames et al., “Ligero: Lightweight Sublinear Arguments Without a Trusted Setup,” Designs, Codes and Cryptography, 2023; ePrint Report 2022/1608. <https://eprint.iacr.org/2022/1608>
- [9] Eli Ben-Sasson et al., “Aurora: Transparent Succinct Arguments for R1CS,” IACR Cryptology ePrint Archive, Report 2018/828. <https://eprint.iacr.org/2018/828>
- [10] Jiaheng Zhang et al., “Transparent Polynomial Delegation and Its Applications to Zero Knowledge Proof,” IACR Cryptology ePrint Archive, Report 2019/1482. <https://eprint.iacr.org/2019/1482>
- [11] Arasu Arun, Srinath Setty, Justin Thaler, “Jolt: SNARKs for Virtual Machines via Lookups,” IACR Cryptology ePrint Archive, Report 2023/1217. <https://eprint.iacr.org/2023/1217>
- [12] RISC Zero, “zkVM Overview,” Developer Docs. <https://dev.risczero.com/api/zkvm/>
- [13] Ariel Gabizon and Zachary J. Williamson, “Plookup: A simplified polynomial protocol for lookup tables,” IACR Cryptology ePrint Archive, Report 2020/315. <https://eprint.iacr.org/2020/315>

索引

AIR, [3](#), [10](#)

Aurora, [12](#)

BBS+ 署名, [7](#)

BBS 署名, [7](#)

Bulletproofs, [8](#)

custom gate, [7](#)

FRI, [10](#)

GKR, [13](#)

Groth16, [5](#)

inner product argument, [8](#)

interactive PCP, [11](#)

IOP, [12](#)

Jolt, [13](#)

KZG, [6](#)

Lasso, [13](#)

layered arithmetic circuit, [3](#), [13](#)

Ligero, [11](#)

lookup, [13](#)

lookup argument, [6](#), [7](#)

Merkle commitment, [10](#)

MPC-in-the-head, [11](#)

One-out-of-Many proof, [9](#)

Pedersen commitment, [8](#)

permutation argument, [6](#), [7](#)

PLONK, [6](#)

Plonkish 制約, [3](#), [6](#)

QAP, [3](#), [5](#)

R1CS, [3](#), [12](#)

range proof, [8](#)

Reed-Solomon 符号, [11](#)

RISC-V, [13](#)

Sigma protocol, [9](#)

STARK, [10](#)

sum-check, [13](#)

trusted setup, [5](#)

unlinkability, [7](#)

Virgo, [13](#)

VM 実行トレース, [3](#)

ZK-STARK, [10](#)

zkVM, [13](#)

ペアリング, [5](#)

ポスト量子性, [10](#)

匿名性, [9](#)

匿名資格証明, [7](#)

多項式委譲, [13](#)

算術化, [3](#)

透明セットアップ, [12](#)

選択的開示, [7](#)

集合所属証明, [9](#)